AWS SAA-C03 Study Guide

Module 1
Foundations + Secure Architectures

Cloud fundamentals • IAM from first principles • VPC networking from scratch • Encryption • Worked practice questions

1. Cloud & AWS Fundamentals (read this first)

1.1 Why cloud exists

Before cloud: to run an application you bought servers, racked them in a data center, over-provisioned for peak traffic, and waited weeks for new hardware. Cloud computing replaces that capital expense with on-demand, pay-as-you-go resources delivered over the internet. The three big wins the exam assumes you internalize: **elasticity** (add/remove capacity in minutes and pay only for what runs), **managed services** (AWS runs the undifferentiated heavy lifting — patching, replication, hardware — so you run less), and **global reach** (deploy near your users without building data centers).

1.2 The AWS global infrastructure

- **Region:** a physical geographic area (us-east-1, eu-west-2...). Regions are fully isolated from each other — an outage in one does not affect another. Most services are Regional: your S3 bucket, VPC, and RDS instance live in exactly one Region. A few are global: IAM, Route 53, CloudFront.
- **Availability Zone (AZ):** one or more physically separate data centers within a Region, with independent power, cooling, and networking, connected to each other by very fast private fiber. Every Region has at least 3 AZs. AZs are far enough apart that a fire or flood takes out only one, but close enough (<~2ms) that synchronous replication between them works.
- **Edge locations:** 600+ small sites worldwide used by CloudFront, Route 53, and Global Accelerator to serve users from nearby, without a full Region.

CORE CONCEPT — The single most important design idea on this exam: Individual machines and even whole data centers WILL fail. AWS gives you AZs and Regions as failure boundaries. You achieve high availability by spreading resources across multiple AZs, and disaster recovery by using multiple Regions. Almost every 'resilient' answer is some form of 'run it in more than one AZ.'

1.3 The Shared Responsibility Model

AWS is responsible for security **OF** the cloud: physical facilities, hardware, the hypervisor, and the managed-service software. You are responsible for security **IN** the cloud: your data, IAM configuration, network rules, OS patching on EC2, and encryption choices. The split moves depending on the service: on EC2 you patch the OS; on RDS, AWS patches it; on Lambda/S3 you only manage your data, code, and permissions.

EXAM TIP: Classification questions: 'physical security of data centers' = AWS. 'Configuring security groups, IAM policies, encrypting data, patching the EC2 guest OS' = customer. 'Patching RDS engine' = AWS.

1.4 The Well-Architected Framework (the exam's skeleton)

The six pillars: **Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability**. The four exam domains map directly onto pillars 2–5. When a question says 'a solutions architect must recommend...', it is asking which design best satisfies one of these pillars under the stated constraints.

2. Identity & Access Management from First Principles

2.1 The problem IAM solves

Every action in AWS — launching an instance, reading an S3 object, even opening the console — is an API call. IAM answers two questions about every single call: **authentication** (who is making this call?) and **authorization** (are they allowed to do this?). Nothing is allowed unless something explicitly allows it — AWS is deny-by-default.

2.2 The identities

- **Root user:** the identity that created the account. It bypasses all IAM policies and cannot be restricted. Rule: enable MFA, lock it away, never use it for daily work, never create access keys for it.
- **IAM user:** a permanent identity with long-term credentials — a password (console) and/or access keys (API/CLI). Meant for a specific human or legacy application. Long-term credentials are the thing attackers steal, which is why modern AWS pushes you away from users toward roles and federation.
- **IAM group:** a bag of users. Groups cannot be nested and are not a 'principal' (you can't reference a group in a resource policy). Attach policies to groups so permissions are managed once, not per person.
- **IAM role:** an identity with permissions but NO permanent credentials. Something must ASSUME the role, and STS hands back temporary credentials (15 min–12 hrs) that expire automatically. Roles are how EC2 instances, Lambda functions, ECS tasks, other accounts, and federated users get permissions. Two policies define a role: the **trust policy** (WHO may assume it) and the **permissions policy** (WHAT they can do once assumed).

CORE CONCEPT — Why roles beat access keys: Temporary credentials expire on their own and are delivered automatically (e.g., via the EC2 instance metadata service). There is nothing to embed in code, nothing to rotate, nothing to leak in a GitHub commit. Any exam scenario with credentials 'stored in the application / AMI / config file' is describing the wrong design; the fix is a role.

2.3 Anatomy of a policy

A policy is a JSON document containing one or more statements. Each statement has: **Effect** (Allow or Deny), **Action** (the API calls, e.g. s3:GetObject), **Resource** (which ARNs it applies to), and optional **Condition** (context tests). Example — read-only access to one bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::my-reports",
      "arn:aws:s3:::my-reports/*"
    ],
    "Condition": {"Bool": {"aws:SecureTransport": "true"}}
  }]
}
```

- Note the two Resource lines: **ListBucket** acts on the bucket itself, **GetObject** acts on objects inside it (/*). Forgetting one of the two is a classic broken-policy question.
- Useful conditions to recognize: `aws:SourceIp` (restrict to office CIDR), `aws:PrincipalOrgID` (only identities from my AWS Organization), `aws:MultiFactorAuthPresent`, `aws:SecureTransport` (require HTTPS).

2.4 How AWS decides Allow or Deny (the evaluation logic)

- Step 1 — Collect every policy that applies to this principal + action + resource: identity policies, resource policies, permissions boundaries, SCPs, session policies.
- Step 2 — Is there an explicit DENY anywhere? If yes → DENIED. Nothing overrides an explicit deny.
- Step 3 — Is there an explicit ALLOW (that isn't cut off by a boundary/SCP)? If yes → ALLOWED.
- Step 4 — Otherwise → implicit deny (the default).

Same-account: an allow in EITHER the identity policy OR the resource policy is enough. **Cross-account:** you need an allow on BOTH sides — account A's identity policy must allow the call, and account B's resource policy (or an assumable role in B) must allow account A in.

2.5 The permission-shaping layers

- **Permissions boundary:** a policy attached to a user/role that sets the MAXIMUM it can ever have. Effective permissions = identity policy ∩ boundary. Used to delegate: 'developers may create roles, but those roles can never exceed this boundary.'
- **Service Control Policies (SCPs):** the same idea at the Organizations level — a ceiling applied to entire accounts/OUs. SCPs never grant anything; identities still need IAM allows within the ceiling. SCPs don't bind the management account.
- **IAM Access Analyzer:** finds resources shared outside your account/org (public buckets, cross-account roles) and can generate least-privilege policies from actual CloudTrail usage.

2.6 Federation: not everyone gets an IAM user

Federation means an external identity provider proves who you are, and AWS trades that proof for temporary role credentials. Map the scenario to the service:

Who is signing in?	Service	How it works
Employees → many AWS accounts / CLI / console	IAM Identity Center (ex AWS SSO)	Connect once to AD or an external IdP (Okta, Entra); assign users/groups 'permission sets' (roles) across all org accounts. The modern default answer.
Employees, legacy direct SAML	IAM SAML federation + STS AssumeRoleWithSAML	Corporate IdP posts a SAML assertion; STS returns role credentials. Appears in older-style questions.
App customers (millions) sign up / sign in	Cognito User Pools	Managed user directory: passwords, MFA, hosted UI, social sign-in; issues JWT tokens your app/API Gateway/ALB validates.
App customers need AWS credentials (e.g., upload straight to S3)	Cognito Identity Pools	Exchanges tokens (from User Pools, Google, Apple, SAML...) for temporary AWS credentials mapped to roles — incl. limited 'guest' access.
Windows workloads needing real Active Directory	AWS Managed Microsoft AD	Actual AD in the cloud; can build a two-way trust with on-prem AD. AD Connector = just a proxy to on-prem AD.

EXAM TIP: Employees → Identity Center. Customers → Cognito. 'AD-aware application' or 'trust with on-prem AD' → Managed Microsoft AD. Never the answer: creating thousands of IAM users.

3. VPC Networking from Scratch

3.1 What a VPC actually is

A Virtual Private Cloud is your own logically isolated slice of the AWS network in one Region — your private data-center network, defined in software. Nothing gets in or out unless you build a door. When you create a VPC you pick a **CIDR block**, the range of private IP addresses it owns.

3.2 CIDR in two minutes

10.0.0.0/16 means: the first 16 bits are fixed (10.0), the remaining 16 bits are yours → 65,536 addresses (10.0.0.0–10.0.255.255). The bigger the /number, the smaller the network: /16 = 65,536 IPs, /24 = 256 IPs, /28 = 16 IPs. VPC CIDRs can be /16 to /28. Plan CIDRs that don't overlap with your other VPCs or your office network — overlapping ranges cannot be peered or routed.

3.3 Subnets: carving up the VPC

A subnet is a slice of the VPC CIDR that lives in exactly ONE Availability Zone. This is the mechanism behind multi-AZ design: to run in 3 AZs, you create (at least) one subnet per AZ. AWS reserves 5 IPs in every subnet (network, router, DNS, future, broadcast) — so a /24 subnet really has 251 usable addresses.

- A **public subnet** is simply a subnet whose route table has a route to an Internet Gateway. A **private subnet** has no such route. That's the whole difference — it's routing, not a checkbox.

3.4 Routing and gateways

- **Route table:** every subnet is associated with exactly one route table; it decides where packets leave. Every table has the un-deletable 'local' route covering the VPC CIDR — all subnets in a VPC can reach each other by default.
- **Internet Gateway (IGW):** the VPC's door to the internet. One per VPC, horizontally scaled and HA by design. For a resource to be internet-reachable it needs ALL of: a public IP, a subnet route 0.0.0.0/0 → IGW, and SG/NACL rules allowing the traffic. Missing any one = 'why can't I reach my instance?' questions.
- **NAT Gateway:** lets PRIVATE subnets initiate outbound internet connections (patches, APIs) while blocking inbound. It's a managed appliance that lives in a PUBLIC subnet with an Elastic IP; private route tables send 0.0.0.0/0 to it. It is AZ-scoped: for HA, deploy one NAT GW per AZ and route each private subnet to the NAT in its own AZ. It bills hourly + per-GB — remember this for cost questions.
- **Egress-only Internet Gateway:** the IPv6 equivalent of NAT (IPv6 addresses are public by nature; this blocks inbound).
- **Elastic IP:** a static public IPv4 address you own and can re-attach; public IPv4 addresses now carry an hourly charge.

3.5 The two firewalls: Security Groups vs Network ACLs

	Security Group	Network ACL
Attaches to	Instance/ENI (the resource)	Subnet (everything in it)
State	STATEFUL — reply traffic is automatically allowed back	STATELESS — you must allow the return path too (ephemeral ports 1024–65535)
Rule types	Allow only (anything not allowed is blocked)	Allow AND Deny, evaluated by rule number, lowest first, first match wins
Can reference	CIDRs, other SGs (e.g., 'allow from the ALB's SG') — this is how tiers are chained	CIDRs only
Default	No inbound, all outbound	Default NACL allows everything
Use for	Primary, per-tier access control	Subnet-level guardrails; the ONLY way to explicitly BLOCK an IP

CORE CONCEPT — Chaining security groups: The standard 3-tier pattern: ALB's SG allows 443 from 0.0.0.0/0. App tier's SG allows the app port only FROM THE ALB'S SG (not a CIDR). Database SG allows 3306 only FROM THE APP TIER'S SG. Instances can scale and change IPs freely — the references follow the groups, not addresses.

3.6 Private connectivity to AWS services and other networks

- **VPC endpoints** keep traffic to AWS services on the AWS network (no IGW/NAT needed). **Gateway endpoints**: S3 & DynamoDB only, added as a route-table entry, FREE. **Interface endpoints** (PrivateLink): an ENI with a private IP in your subnet fronting nearly any AWS service or a third party's service; billed hourly + per-GB. Endpoint policies restrict what identities/resources can be reached through them.
- **VPC peering:** private 1:1 link between two VPCs (same/different account or Region). Non-transitive: A–B and B–C gives A no path to C. CIDRs must not overlap. Fine for a handful of VPCs; a mesh of N VPCs needs N(N-1)/2 peerings — unmanageable at scale.
- **Transit Gateway:** a Regional hub router. Attach VPCs, VPNs, and Direct Connect; everything routes through the hub (transitive). The answer for 'dozens of VPCs + on-prem' and for hub-and-spoke network architectures. Supports inter-Region peering.
- **Site-to-Site VPN:** IPsec tunnels over the public internet between your router (customer gateway) and AWS (virtual private gateway or TGW). Quick (hours), encrypted, ~1.25 Gbps/tunnel, but internet-variable latency.

- **Direct Connect:** a dedicated physical fiber link into AWS. Consistent latency and high bandwidth, private — but takes weeks+ to provision and is NOT encrypted by default (run a VPN over it, or MACsec, if encryption is required).
- **VPC Flow Logs:** metadata (accept/reject, IPs, ports) for traffic at VPC/subnet/ENI level → CloudWatch Logs or S3. The troubleshooting and forensics answer for 'why is this connection failing / who talked to what.'

3.7 Walkthrough: reading a standard VPC

A production VPC 10.0.0.0/16 across 3 AZs typically has 6 subnets: three public (10.0.0.0/24, 10.0.1.0/24, 10.0.2.0/24 — route 0.0.0.0/0 → IGW) holding the ALB nodes and NAT Gateways, and three private (10.0.10.0/24, 10.0.11.0/24, 10.0.12.0/24 — route 0.0.0.0/0 → NAT-in-my-AZ) holding app instances and databases. Add a Gateway endpoint for S3 so app-to-S3 traffic skips the NAT (free and private). This picture is behind a huge fraction of exam questions — be able to draw it.

EXAM TIP: 'Instance in a private subnet must download OS patches' → NAT Gateway. 'Must access S3 privately/cheaply' → Gateway endpoint. 'Reach a vendor's service privately without peering' → PrivateLink interface endpoint. '40 VPCs and on-prem must all communicate' → Transit Gateway.

4. Encryption & Data Protection from First Principles

4.1 The two kinds of encryption

Symmetric encryption uses one key for both encrypt and decrypt (AES-256) — fast, used for actual data. **Asymmetric** uses a public/private key pair (RSA) — slower, used for signing and key exchange. You must also distinguish encryption **at rest** (data on disk: EBS, S3, RDS) from **in transit** (data moving: TLS/HTTPS).

4.2 KMS and envelope encryption

AWS KMS stores and controls **KMS keys** that never leave the service unencrypted — you send data to KMS APIs, you never hold the key. Because KMS only directly encrypts up to 4 KB, real data uses **envelope encryption**: (1) call GenerateDataKey → KMS returns a plaintext data key + the same key encrypted under your KMS key; (2) encrypt your data locally with the plaintext data key, then discard it; (3) store the encrypted data key alongside the data. To decrypt, send the encrypted data key to KMS, get the plaintext key back, decrypt locally. Every AWS service's 'encrypted with KMS' checkbox (EBS, S3 SSE-KMS, RDS) is doing exactly this under the hood.

- Key types: **AWS-managed** (aws/s3 etc. — free, no control, auto-rotated), **customer-managed** (~$1/mo — you control the key policy, enable/disable, optional yearly rotation, cross-account grants, CloudTrail-audited usage). Choose customer-managed whenever the scenario says 'control', 'audit key usage', or 'cross-account'.
- **Key policies** are resource policies on the key itself — the primary gate. Cross-account decryption requires the key policy to allow the external principal AND that principal's IAM policy to allow kms:Decrypt.
- **CloudHSM**: dedicated, single-tenant FIPS 140-2 Level 3 hardware where YOU hold the keys — only for strict regulatory demands; KMS satisfies almost everything else.

4.3 Applying it per service

- **S3:** SSE-S3 (default, S3-owned keys) → SSE-KMS (auditability + access control; enable Bucket Keys to cut KMS request costs) → SSE-C (you supply the key per request) → client-side (AWS never sees plaintext). Enforce encryption or TLS with bucket-policy conditions.
- **EBS/RDS:** encrypt at creation; snapshots and copies inherit encryption. To encrypt an existing unencrypted volume/DB: snapshot → copy the snapshot WITH encryption → restore. You cannot flip encryption on in place.
- **Secrets:** Secrets Manager = secrets with automatic rotation (native RDS support) and cross-Region replication, per-secret cost. SSM Parameter Store SecureString = cheap/free config secrets, NO native rotation. 'Rotate automatically' → Secrets Manager, always.

- **Certificates:** ACM issues free public TLS certs, auto-renews, and deploys to ALB/CloudFront/API Gateway. CloudFront certs must live in us-east-1. You can't export ACM public certs (so not for EC2 directly).

5. Reference Architecture: Secure 3-Tier Web Application

Trace this end to end — half of Domain 1's questions are variations of it. **Flow:** User → Route 53 → CloudFront (+WAF) → ALB (public subnets, 3 AZs) → EC2 Auto Scaling group (private subnets) → RDS Multi-AZ (private DB subnets).

- **Edge:** Route 53 alias record → CloudFront. WAF on CloudFront blocks SQLi/XSS and rate-limits. ACM cert gives HTTPS. Shield Standard absorbs basic DDoS automatically.
- **Load balancer:** ALB in the three public subnets; SG allows 443 from the internet (or from CloudFront prefix list only, to prevent bypassing the CDN). HTTP→80 redirect to HTTPS.
- **App tier:** instances in private subnets; SG allows app port only from the ALB SG. No public IPs. Instance role grants exactly the S3/DynamoDB/SQS access the app needs. Admin access via SSM Session Manager — no SSH keys, no bastion, no port 22, all sessions logged.
- **Data tier:** RDS in private subnets, SG allows 3306 from app SG only, storage encrypted with a customer-managed KMS key, credentials in Secrets Manager with rotation; app fetches them at runtime via its role.
- **Egress & endpoints:** NAT gateway per AZ for patches; Gateway endpoint for S3. VPC Flow Logs + CloudTrail + GuardDuty on across the account.

Every 'make it more secure' question is asking you to find the layer of this diagram that's mis-built: public DB subnet, hardcoded keys, SG open to 0.0.0.0/0 on the app tier, unencrypted storage, SSH from anywhere.

6. Worked Practice Questions

Practice Q1. An application on EC2 instances must read and write objects in one S3 bucket. The security team requires no long-term credentials on the instances. What should the architect do?

✗ Store access keys in the application's config file / user data — long-term credentials, exactly what's forbidden, leak-prone.

✗ Create an IAM user per instance — still long-term keys, unmanageable.

✗ Make the bucket public — removes authentication entirely; never correct.

✓ Create an IAM role with a least-privilege S3 policy and attach it via instance profile. The instance gets auto-rotated temporary credentials from instance metadata. Roles-over-keys is the single most repeated pattern on the exam.

Practice Q2. A company must let instances in private subnets access DynamoDB. Security requires traffic never traverses the internet, and cost must be minimized. Which solution?

✗ NAT Gateway — works, but traffic exits toward public endpoints and NAT bills per-GB: fails both stated requirements relative to the alternative.

✗ Interface VPC endpoint — private, yes, but hourly + per-GB charges; DynamoDB has a cheaper native option.

✓ Gateway VPC endpoint for DynamoDB — free, private, a route-table entry. Gateway endpoints exist for exactly two services: S3 and DynamoDB. When one of those two + 'private' + 'cheap' appears, this is the answer.

Practice Q3. Auditors require that a specific malicious IP address be blocked from reaching an application's subnet. What accomplishes this?

✗ Security group deny rule — SGs have no deny rules; they only allow.

✗ Remove the SG allow rule — would block everyone, not one IP.

✓ Network ACL inbound DENY rule for that IP with a low rule number on the subnet. NACLs are the only VPC-native explicit-block mechanism. (At the edge, WAF IP sets also block — pick NACL when the question is subnet-scoped.)

Practice Q4. An application stores database credentials that compliance says must rotate every 30 days automatically with no code deployed for rotation. Best fit?

✗ SSM Parameter Store SecureString — encrypted, but no native rotation.

✗ Encrypt credentials in a config file with KMS — rotation is entirely manual.

✓ AWS Secrets Manager with managed rotation — native scheduled rotation for RDS and others. 'Automatic rotation' is the trigger phrase; Parameter Store is the answer only when the ask is 'free/low-cost storage of config/secrets' without rotation.

Practice Q5. A company runs workloads in 40 AWS accounts under AWS Organizations. Security wants to guarantee that NO identity in any member account can disable CloudTrail, including account administrators. What should they use?

✗ IAM policies in each account denying cloudtrail:StopLogging — admins in each account could edit or detach them.

✗ Permissions boundary — applies per-identity and is also editable by account admins.

✓ A Service Control Policy on the org root/OUs denying cloudtrail:StopLogging and DeleteTrail. SCPs are ceilings applied above the account; member-account admins cannot override them. 'Prevent even admins in member accounts' = SCP, every time.

Practice Q6. Employees authenticate to a corporate identity provider and need to access the AWS consoles of 15 accounts with different permission levels. The company wants centralized management. What's the best solution?

✗ IAM users in each account — 15×N credentials to manage, no central control.

✗ Cognito — built for application end-users, not workforce access to AWS accounts.

✓ IAM Identity Center connected to the corporate IdP, with permission sets assigned to groups across all accounts. Workforce + multiple accounts + SSO → Identity Center.

Practice Q7. An architect must ensure all objects in an S3 bucket are encrypted with keys the company controls, and every use of the key must be auditable. Which configuration?

✗ SSE-S3 — encrypted, but AWS-owned keys: no key policy, no per-key audit trail.

✗ Client-side encryption — maximal control but heavy operational burden; not required by the scenario.

✓ SSE-KMS with a customer-managed key — key policies control use, every Encrypt/Decrypt lands in CloudTrail. 'Control + audit' maps to customer-managed KMS.

Self-check before moving to Module 2

- Can you explain why a subnet is 'public'? (Route to IGW — nothing else.)
- Can you walk the 4-step policy evaluation and say what wins? (Explicit deny.)
- Can you draw the 3-tier VPC with SG chaining from memory?
- Do you know the two Gateway-endpoint services cold? (S3, DynamoDB.)
- Roles vs users vs Identity Center vs Cognito — can you place each in one sentence?